

Scenario

We are continuing to build the application for a **training institute**.

In Week 1 we built the first working version of the Training Institute Management System. The application can now:

- Create, view, edit and delete Students
- Create, view, edit and delete Courses
- Validate form input
- Store the information in MySQL

Let's Start with Week 2

Week 2 focuses on organizing the application we built in Week 1.

At this stage, we are **not adding new features**. The application should behave exactly as it does today, but it will get a navigation bar and a footer.

Instead, we want to focus on the simple question:

How can we organize a web application so that we never write the same code twice?

To answer this question, we need to understand:

1. MVC Architecture
2. Controllers and Resource Routes
3. Layouts and Partial
4. Blade Components
5. The DRY principle (Don't Repeat Yourself)

Problem Definition

The Week 1 application works, but it is hard to maintain.

Every request is handled by a closure inside `routes/web.php`. If we continue building applications in this format we will quickly have a huge `web.php` file.

The other problem is we have to add a navigation bar and footer to the application. This navigation bar and footer are going to be repeated in every page. We want to organize our view so that we can use layouts to structure the view properly.

Problem	Where you can see it in Week 1
Everything in one file	All Student routes, validation and queries are inside <code>routes/web.php</code>

Problem	Where you can see it in Week 1
Duplicated validation	The create and update routes repeat almost the same rules
Hard-coded URLs	Views contain links such as <code>/students/create</code>
Repeated page HTML	Every view repeats <code><!DOCTYPE html></code> , <code><head></code> and the error block
Duplicated forms	The create and edit forms are almost identical
Repeated buttons	Every list page repeats the same delete form

In Week 2 we **refactor** the application. We keep the same behavior and apply two ideas:

- **MVC:** every piece of code lives in the layer where it belongs.
- **DRY (Don't Repeat Yourself):** every piece of code exists in **one place only**. If it changes, you change it once.

Goal & Deliverables

By the end of Week 2, your application should behave like Week 1, but be organized using MVC and DRY:

- `StudentController` and `CourseController` handle all request logic
- `routes/web.php` contains only a few lines and no business logic
- Named routes are used instead of hard-coded URLs
- Validation rules are written once in each controller
- All pages share one layout with a **navigation bar** and a **footer**
- Reusable **Blade components** for alerts, form fields and delete buttons
- One shared form partial for create and edit

Week 2 Checklist

By the end of the workshop, you should be able to:

- Create controllers using Artisan
- Use `Route::resource()`, named routes and route model binding
- Create a layout with a navbar and a footer
- Create reusable Blade components with `@props`
- Reuse one form partial for create and edit
- Decide which layer a piece of code belongs to

What is MVC?

MVC (Model–View–Controller) splits the application into three parts, each with one job.

Layer	Responsibility	Location
Model	The data and the database	<code>app/Models/</code>
View	The HTML the user sees	<code>resources/views/</code>
Controller	Receives the request, asks the Model for data, returns a View	<code>app/Http/Controllers/</code>

Routes are not one of the three letters. A route only maps a URL to a controller method.

A request travels like this: **Browser** → **Route** → **Controller** → **Model** → **Controller** → **View** → **Browser**.

What is DRY?

DRY means **Don't Repeat Yourself**. If you copy and paste the same code into many files, a small change has to be made in every copy, and you will forget one.

Blade gives us three tools to avoid repetition:

Tool	Use it for	Example in this workshop
Layout (<code>@extends</code>)	The page skeleton shared by all pages	<code>layouts/app.blade.php</code>
Partial (<code>@include</code>)	A fixed chunk of HTML	<code>partials/navbar.blade.php</code>
Component (<code><x-...></code>)	A reusable piece that takes inputs	<code><x-form-input name="email" /></code>

Step 1: Prepare the Workshop 2 Branch

Workshop 2 builds on top of the Workshop 1 code, so first make sure Workshop 1 is complete and committed.

Open your WSL terminal and go to your workshop repository:

```
cd ~/6CS056/6CS056-Workshop
git switch Workshop1
git status
```

`git status` should say there is nothing to commit. If not, commit your Workshop 1 work first.

Now create the Workshop 2 branch from it:

```
git switch -c Workshop2
```

Start the server and confirm that Student CRUD still works before we change anything:

```
php artisan serve
```

Checkpoint: If something breaks later, you know it was your refactoring and not the original code.

Step 2: Create the Student Controller

A controller is a PHP class. Each public method handles one kind of request. Create it with Artisan:

```
php artisan make:controller StudentController
```

Open the file and add the imports that we will need:

`app/Http/Controllers/StudentController.php`

```
<?php

namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;

class StudentController extends Controller
{
    //
}
```

Every CRUD operation from Week 1 will become one method of this class. Do **not** delete the closure routes yet. The application keeps working while we build the controller.

Step 3: Move the Read Logic (index and show)

Inside the `StudentController` class, add these two methods. The code is the same as the Week 1 closures, only the location has changed.

```
public function index()
{
    $students = Student::all();

    return view('student.list', [
        'students' => $students,
    ]);
}
```

```

}

public function show($id)
{
    $student = Student::findOrFail($id);

    return view('student.detail', [
        'student' => $student,
    ]);
}

```

The controller asks the **Model** for data (`Student::all()`) and hands it to the **View**. It does not build any HTML itself.

Step 4: Move the Create Logic (create and store)

Add two more methods. `create` shows the empty form and `store` saves it:

```

public function create()
{
    return view('student.create');
}

public function store(Request $request)
{
    $validated = $request->validate([
        'name'          => 'required|string|max:255',
        'email'          => 'required|email|max:255|unique:students,email',
        'phone'          => 'required|string|max:20',
        'address'         => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date',
    ]);

    $student = Student::create($validated);

    return redirect()
        ->route('students.index')
        ->with('success', "Student {$student->name} created successfully!");
}

```

```
}
```

Note: We added `unique:students,email`. In Week 1 a duplicate email passed validation and caused a database error. The route name `students.index` is defined in Step 7.

Step 5: Move the Update Logic (edit and update)

Add two more methods. `edit` shows the form filled with the existing data and `update` saves the changes:

```
public function edit($id)
{
    $student = Student::findOrFail($id);

    return view('student.edit', [
        'student' => $student,
    ]);
}

public function update(Request $request, $id)
{
    $student = Student::findOrFail($id);

    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => [
            'required', 'email', 'max:255',
            Rule::unique('students')->ignore($student->id),
        ],
        'phone' => 'required|string|max:20',
        'address' => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date',
    ]);

    $student->update($validated);

    return redirect()
        ->route('students.show', $student->id)
        ->with('success', 'Student updated successfully!');
```

```
}
```

Note: In Week 1 the `Rule` class was used without being imported. That is why we added `use Illuminate\Validation\Rule;` at the top of the controller.

Step 6: Move the Delete Logic (destroy)

Add the last method:

```
public function destroy($id)
{
    $student = Student::findOrFail($id);
    $student->delete();

    return redirect()
        ->route('students.index')
        ->with('success', 'Student deleted successfully!');
}
```

Checkpoint: Your `StudentController` now has seven methods: `index`, `create`, `store`, `show`, `edit`, `update` and `destroy`. Compare each one with the matching closure in `routes/web.php`. The logic should be the same.

Step 7: Connect Routes to the Controller

Now we replace the closures with routes that point to the controller. Open:

`routes/web.php`

Delete every Student route from Week 1 (all the `Route::get`, `Route::post`, `Route::put` and `Route::delete` closures for students). Keep the default welcome route.

Make sure the file starts with these imports:

```
<?php

use App\Http\Controllers\StudentController;
use Illuminate\Support\Facades\Route;
```

Add the Student routes:

```
Route::get('/students', [StudentController::class, 'index'])
    ->name('students.index');

Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');

Route::post('/students', [StudentController::class, 'store'])
    ->name('students.store');

Route::get('/students/{id}', [StudentController::class, 'show'])
    ->name('students.show');

Route::get('/students/{id}/edit', [StudentController::class, 'edit'])
    ->name('students.edit');

Route::put('/students/{id}', [StudentController::class, 'update'])
    ->name('students.update');

Route::delete('/students/{id}', [StudentController::class, 'destroy'])
    ->name('students.destroy');
```

- `>name('students.index')` gives the route a name, so we can write `route('students.index')` instead of typing `/students`.

Watch out: `/students/create` must be defined **before** `/students/{id}`. Otherwise Laravel treats the word `create` as an `{id}`. Also, Week 1 used `/student/create` (singular). From now on all URLs use the plural `/students/...`.

Refresh `http://127.0.0.1:8000/students` and test the application. It should behave exactly as before.

Step 8: Simplify with Route::resource()

The seven routes above follow a standard pattern. Laravel can generate all of them with one line. Replace the seven Student routes with:

```
Route::resource('students', StudentController::class);
```

Check what Laravel created:

```
php artisan route:list --except-vendor
```

Method	URI	Controller method	Route name
GET	/students	index	students.index
GET	/students/create	create	students.create
POST	/students	store	students.store
GET	/students/{student}	show	students.show
GET	/students/{student}/edit	edit	students.edit
PUT/PATCH	/students/{student}	update	students.update
DELETE	/students/{student}	destroy	students.destroy

The URLs and names are the same as Step 7, so nothing changes in the application. The only difference is that the parameter is now called `{student}`.

Step 9: Use Named Routes

Hard-coded URLs break when a URL changes. Named routes are defined in one place and every link follows them.

In Blade, always use `route()`:

Instead of	Write
/students	<code>{{ route('students.index') }}</code>
/students/create	<code>{{ route('students.create') }}</code>
/students/{{ \$student->id }}	<code>{{ route('students.show', \$student) }}</code>
/students/{{ \$student->id }}/edit	<code>{{ route('students.edit', \$student) }}</code>
form action to save	<code>{{ route('students.store') }}</code>
form action to update	<code>{{ route('students.update', \$student) }}</code>
form action to delete	<code>{{ route('students.destroy', \$student) }}</code>

You do not need to edit the old views one by one. All Student views are rewritten in Step 14 using `route()`.

Step 10: Route Model Binding

Four controller methods start with the same line:

```
$student = Student::findOrFail($id);
```

Laravel can do this for you. If a controller parameter is typed as a model and its name matches the route parameter

(`{student}`), Laravel finds the record automatically, and returns a 404 page if it does not exist.

Update the four methods in `StudentController`:

```
public function show(Student $student)
{
    return view('student.detail', [
        'student' => $student,
    ]);
}

public function edit(Student $student)
{
    return view('student.edit', [
        'student' => $student,
    ]);
}

public function update(Request $request, Student $student)
{
    $validated = $request->validate([
        'name'          => 'required|string|max:255',
        'email'         => [
            'required', 'email', 'max:255',
            Rule::unique('students')->ignore($student->id),
        ],
        'phone'         => 'required|string|max:20',
        'address'        => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date',
    ]);

    $student->update($validated);

    return redirect()
        ->route('students.show', $student)
        ->with('success', 'Student updated successfully!');
}

public function destroy(Student $student)
```

```
{
    $student->delete();

    return redirect()
        ->route('students.index')
        ->with('success', 'Student deleted successfully!');
}
```

The parameter name in the method (`$student`) must match the route parameter (`{student}`). Visit </students/9999> to see the 404 page.

Step 11: Write the Validation Rules Once

The validation rules are written twice, in `store` and in `update`. This is the **DRY** problem from the start of the workshop. We fix it by moving the rules into **one private method** of the controller.

The only difference between the two cases is the email rule. When updating, the student's own email must be ignored by the `unique` check. We handle this with an optional `$student` parameter.

Add this method at the bottom of `StudentController`:

```
private function rules(?Student $student = null): array
{
    return [
        'name'          => 'required|string|max:255',
        'email'         => [
            'required', 'email', 'max:255',
            Rule::unique('students')->ignore($student?->id),
        ],
        'phone'         => 'required|string|max:20',
        'address'       => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date',
    ];
}
```

Then use it in `store` and `update`. This is the complete `StudentController`:

`app/Http/Controllers/StudentController.php`

```
<?php
```

```

namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;

class StudentController extends Controller
{
    public function index()
    {
        return view('student.list', [
            'students' => Student::all(),
        ]);
    }

    public function create()
    {
        return view('student.create');
    }

    public function store(Request $request)
    {
        $student = Student::create($request->validate($this->rules()));

        return redirect()
            ->route('students.index')
            ->with('success', "Student {$student->name} created successfully!");
    }

    public function show(Student $student)
    {
        return view('student.detail', ['student' => $student]);
    }

    public function edit(Student $student)
    {
        return view('student.edit', ['student' => $student]);
    }
}

```

```

    }

    public function update(Request $request, Student $student)
    {
        $student->update($request->validate($this->rules($student)));

        return redirect()
            ->route('students.show', $student)
            ->with('success', 'Student updated successfully!');
    }

    public function destroy(Student $student)
    {
        $student->delete();

        return redirect()
            ->route('students.index')
            ->with('success', 'Student deleted successfully!');
    }

    private function rules(?Student $student = null): array
    {
        return [
            'name'          => 'required|string|max:255',
            'email'          => [
                'required', 'email', 'max:255',
                Rule::unique('students')->ignore($student?->id),
            ],
            'phone'          => 'required|string|max:20',
            'address'        => 'nullable|string|max:500',
            'date_of_birth' => 'nullable|date',
        ];
    }
}

```

`$request->validate(...)` returns only the validated fields, so it can be passed straight to `create()` and `update()`. If validation fails, Laravel redirects back with the errors and the old input. `$student?->id` is `null` when creating, so nothing is ignored.

Step 12: Give the Model Some Responsibility

The Model is the right place for rules about the data itself, such as how a date is represented or how an age is calculated.

`app/Models/Student.php`

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = [
        'name',
        'email',
        'phone',
        'address',
        'date_of_birth',
    ];

    protected function casts(): array
    {
        return [
            'date_of_birth' => 'date',
        ];
    }

    protected function age(): Attribute
    {
        return Attribute::get(fn () => $this->date_of_birth->age);
    }
}
```

- The **cast** makes `date_of_birth` a date object, so we can write `>format('d M Y')`.
- The **accessor** creates a virtual attribute, so `$student->age` works even though there is no `age` column.

Because of the cast, a date input needs `->format('Y-m-d')`. The views in Step 14 already handle this.

Step 13: Component-Based Blade Layouts

In this step, we replace repeated page markup with **Blade components**. Instead of repeating same HTML content in each page, pages will be composed with component tags such as `<x-layouts.app>` and `<x-navbar />`.

A component can encapsulate markup and accept inputs (props) or page content (a slot). This makes the structure explicit and keeps reusable UI in one place.

Repeated code	Reusable component	How it is used
HTML document and page shell	<code>components/layouts/app.blade.php</code>	<code><x-layouts.app title="Students">...</x-layouts.app></code>
Navigation links	<code>components/navbar.blade.php</code>	<code><x-navbar /></code>
Footer	<code>components/footer.blade.php</code>	<code><x-footer /></code>
Success message	<code>components/alert.blade.php</code>	<code><x-alert /></code>
Label, input and validation error	<code>components/form-input.blade.php</code>	<code><x-form-input ... /></code>
Label, textarea and validation error	<code>components/form-textarea.blade.php</code>	<code><x-form-textarea ... /></code>
Delete form	<code>components/delete-form.blade.php</code>	<code><x-delete-form ... /></code>
Shared Student fields	<code>components/students/form.blade.php</code>	<code><x-students.form ... /></code>

Create these files (folder names must match):

```
public/css/app.css
resources/views/components/layouts/app.blade.php
resources/views/components/navbar.blade.php
resources/views/components/footer.blade.php
resources/views/components/alert.blade.php
resources/views/components/delete-form.blade.php
resources/views/components/form-input.blade.php
resources/views/components/form-textarea.blade.php
resources/views/components/students/form.blade.php
```

13.1 Stylesheet

```
* { box-sizing: border-box; }

body {
  font-family: Arial, sans-serif;
  margin: 0;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.navbar {
  background: #1f2937;
  padding: 14px 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.navbar a { color: #ffffff; text-decoration: none; margin-left: 20px; }
.navbar .brand { font-weight: bold; font-size: 18px; margin-left: 0; }
.navbar a.active { border-bottom: 2px solid #ffffff; }

.container { flex: 1; width: 100%; max-width: 960px; margin: 0 auto; padding: 24px; }

.footer { background: #f3f4f6; color: #555555; text-align: center; padding: 16px; font-size: 14px; }

.btn {
  background: #2563eb; color: #ffffff; border: 0; border-radius: 4px;
  padding: 8px 14px; text-decoration: none; cursor: pointer; font-size: 14px;
}

.alert { padding: 10px 14px; margin-bottom: 16px; border-radius: 4px; }
.alert-success { background: #e6f4ea; color: #137333; }

table { border-collapse: collapse; width: 100%; margin-top: 16px; }
```

```
th, td { border: 1px solid #cccccc; padding: 8px; text-align: left; }

.form-group { margin-bottom: 16px; }
.form-group label { display: block; margin-bottom: 4px; }
.form-group input[type="text"],
.form-group input[type="email"],
.form-group input[type="date"],
.form-group input[type="number"],
.form-group select,
.form-group textarea { width: 100%; max-width: 400px; padding: 6px; }
.form-group.checkbox label { display: inline; }
.error { color: #c5221f; font-size: 14px; margin: 4px 0 0; }

.inline-form { display: inline; }
```

13.2 Navigation and footer components

Blade components are anonymous and are rendered with component tags.

[resources/views/components/navbar.blade.php](#)

```
<nav class="navbar">
    <a href="{{ url('/') }}" class="brand">Training Institute</a>

    <div>
        <a href="{{ route('students.index') }}"
            class="{{ request()->routeIs('students.*') ? 'active' : '' }}">
            Students
        </a>
    </div>
</nav>
```

`request()->routeIs('students.*')` is true on every Student page, so the current link is underlined automatically. Add the Courses link in Step 15.

[resources/views/components/footer.blade.php](#)

```
<footer class="footer">
    <p>&copy; {{ date('Y') }} Training Institute Management System &middot;</p>
```

```
</footer>
```

13.3 Reusable UI components

A component file's name determines its tag. For example, `components/alert.blade.php` is rendered with `<x-alert />`. `@props` declares named inputs, while `$attributes` forwards additional HTML attributes to the underlying element.

`resources/views/components/alert.blade.php`

```
@props(['type' => 'success'])

@if(session($type))
    <div class="alert alert-{{ $type }}">
        {{ session($type) }}
    </div>
@endif
```

`resources/views/components/delete-form.blade.php`

```
@props(['action'])

<form action="{{ $action }}" method="POST" class="inline-form"
    onsubmit="return confirm('Are you sure you want to delete this record?')">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn">Delete</button>
</form>
```

`resources/views/components/form-input.blade.php`

```
@props(['name', 'label', 'type' => 'text', 'value' => ''])

<div class="form-group">
    <label for="{{ $name }}">{{ $label }}</label>
    <input type="{{ $type }}" id="{{ $name }}" name="{{ $name }}"
        value="{{ old($name, $value) }}" {{ $attributes }}>
    @error($name)
        <p class="error">{{ $message }}</p>
    @enderror
</div>
```

```
@enderror

</div>
```

<resources/views/components/form-textarea.blade.php>

```
@props(['name', 'label', 'value' => ''])

<div class="form-group">
    <label for="{{ $name }}">{{ $label }}</label>
    <textarea id="{{ $name }}" name="{{ $name }}" {{ $attributes }}>{{ old($name,
$value) }}</textarea>
    @error($name)
        <p class="error">{{ $message }}</p>
    @enderror
</div>
```

13.4 The layout component

The layout component owns the HTML document, shared navigation, success alert, and footer. The page-specific content is supplied through Laravel's default **slot**. The `title` is a component prop.

<resources/views/components/layouts/app.blade.php>

```
@props(['title' => 'Home'])

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{{ $title }} | Training Institute</title>
    <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>
    <x-navbar />

    <main class="container">
        <x-alert />
        {{ $slot }}
    </main>
</body>
</html>
```

```
</main>

<x-footer />

</body>

</html>
```

A page now wraps its content in `<x-layouts.app title="..."> ... </x-layouts.app>`. This replaces the traditional `@extends('layouts.app')`, `@section('title', ...)`, `@section('content')`, and `@endsection` pattern.

Step 14: Rewrite the Student Views to Use Components

All Student pages will now use the layout component. The shared Student fields are also made into a component, so create and edit pages do not need to repeat the same fields.

14.1 Student list

`resources/views/student/list.blade.php`

```
<x-layouts.app title="Students">
    <h1>Students</h1>
    <a href="{{ route('students.create') }}" class="btn">Create Student</a>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Email</th>
                <th>Phone</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @forelse($students as $student)
                <tr>
                    <td>{{ $student->id }}</td>
                    <td>{{ $student->name }}</td>
```

```

        <td>{{ $student->email }}</td>
        <td>{{ $student->phone }}</td>
        <td>
            <a href="{{ route('students.show', $student) }}">View</a>
            <a href="{{ route('students.edit', $student) }}">Edit</a>
            <x-delete-form :action="route('students.destroy', $student)" />
        </td>
    </tr>
    @empty
    <tr>
        <td colspan="5">No students found.</td>
    </tr>
    @endforelse
</tbody>
</table>
</x-layouts.app>

```

14.2 Shared Student form component

[resources/views/components/students/form.blade.php](#)

```

@props(['student'])

<x-form-input name="name" label="Name" :value="$student->name" />

<x-form-input name="email" label="Email" type="email" :value="$student->email" />

<x-form-input name="phone" label="Phone" :value="$student->phone" />

<x-form-textarea name="address" label="Address" :value="$student->address" />

<x-form-input name="date_of_birth" label="Date of Birth" type="date"
    :value="$student->date_of_birth?->format('Y-m-d')" />

```

The component expects a `Student` object. The create page passes a new empty model; the edit page passes the existing model. A colon before an attribute (such as `:student="$student"`) means the value is a PHP expression rather than a literal string.

14.3 Create Student

`resources/views/student/create.blade.php`

```
<x-layouts.app title="Create Student">
    <h1>Create Student</h1>

    <form action="{{ route('students.store') }}" method="POST">
        @csrf

        <x-students.form :student="new \App\Models\Student()" />

        <button type="submit" class="btn">Create Student</button>
        <a href="{{ route('students.index') }}">Cancel</a>
    </form>
</x-layouts.app>
```

14.4 Edit Student

`resources/views/student/edit.blade.php`

```
<x-layouts.app title="Edit Student">
    <h1>Edit Student</h1>

    <form action="{{ route('students.update', $student) }}" method="POST">
        @csrf
        @method('PUT')

        <x-students.form :student="$student" />

        <button type="submit" class="btn">Update Student</button>
        <a href="{{ route('students.show', $student) }}">Cancel</a>
    </form>
</x-layouts.app>
```

14.5 Student details

`resources/views/student/detail.blade.php`

```

<x-layouts.app title="Student Details">
    <h1>Student Details</h1>

    <p><strong>ID:</strong> {{ $student->id }}</p>
    <p><strong>Name:</strong> {{ $student->name }}</p>
    <p><strong>Email:</strong> {{ $student->email }}</p>
    <p><strong>Phone:</strong> {{ $student->phone }}</p>
    <p><strong>Address:</strong> {{ $student->address }}</p>
    <p><strong>Date of Birth:</strong> {{ $student->date_of_birth?->format('d M Y') }}
</p>

    <p><strong>Age:</strong> {{ $student->age ?? 'Not available' }}</p>

    <a href="{{ route('students.edit', $student) }}" class="btn">Edit Student</a>
    <a href="{{ route('students.index') }}">Back to Students</a>
</x-layouts.app>

```

Checkpoint

- Confirm that all component files exist under `resources/views/components`.
- Open `/students`; the navbar, footer, and success alert should appear through the layout component.
- Open the create and edit pages and confirm that the same Student fields render on both.
- Confirm validation errors and old input values still appear after a failed submission.

Key idea: component-based layouts compose a page from reusable components and slots. The layout controls the shared page shell, while each page supplies its own content. Small components encapsulate repeated interface elements and receive data through props.

Step 15: Apply the Same Pattern to Courses

Now repeat the process for Courses. Because the reusable pieces already exist, this step is much shorter than the Student work. Let Artisan generate the files:

```
php artisan make:controller CourseController --resource --model=Course
```

This creates `CourseController` with the seven resource methods, already typed with `Course $course`.

15.1 Route

Add the import and the route in `routes/web.php`:

```
use App\Http\Controllers\CourseController;

Route::resource('courses', CourseController::class);
```

15.2 Model

app/Models/Course.php

```
protected $fillable = [
    'name',
    'description',
    'duration',
    'fee',
    'difficulty',
    'is_active',
];

protected function casts(): array
{
    return [
        'is_active' => 'boolean',
        'fee'       => 'decimal:2',
    ];
}
```

15.3 Controller

Replace the generated methods with the code below. The validation rules are written once in the private `rules()` method, exactly like the Student controller. The values in `Rule::in(...)` must match the `difficulty` values of your Week 1 migration.

app/Http/Controllers/CourseController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Course;
```

```

use Illuminate\Http\Request;
use Illuminate\Validation\Rule;

class CourseController extends Controller
{
    public function index()
    {
        return view('course.list', ['courses' => Course::all()]);
    }

    public function create()
    {
        return view('course.create');
    }

    public function store(Request $request)
    {
        $course = Course::create($request->validate($this->rules()));

        return redirect()
            ->route('courses.index')
            ->with('success', "Course {$course->name} created successfully!");
    }

    public function show(Course $course)
    {
        return view('course.detail', ['course' => $course]);
    }

    public function edit(Course $course)
    {
        return view('course.edit', ['course' => $course]);
    }

    public function update(Request $request, Course $course)
    {
        $course->update($request->validate($this->rules()));
    }
}

```

```

        return redirect()
            ->route('courses.show', $course)
            ->with('success', 'Course updated successfully!');
    }

    public function destroy(Course $course)
    {
        $course->delete();

        return redirect()
            ->route('courses.index')
            ->with('success', 'Course deleted successfully!');
    }

    private function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'description' => 'nullable|string',
            'duration' => 'required|integer|min:1',
            'fee' => 'required|numeric|min:0',
            'difficulty' => ['required', Rule::in(['Easy', 'Medium', 'Hard'])],
            'is_active' => 'boolean',
        ];
    }
}

```

15.4 Views

Create the four views in `resources/views/course/`. They are short because the layout, the form fields and the delete button are reused.

`resources/views/course/_form.blade.php`

```

@php
    $course = $course ?? new \App\Models\Course(['is_active' => true]);
@endphp

```

```

<x-form-input name="name" label="Name" :value="$course->name" />

<x-form-textarea name="description" label="Description" :value="$course->description"
/>

<x-form-input name="duration" label="Duration (weeks)" type="number" min="1"
:value="$course->duration" />

<x-form-input name="fee" label="Fee" type="number" step="0.01" min="0"
:value="$course->fee" />

<div class="form-group">
    <label for="difficulty">Difficulty</label>
    <select id="difficulty" name="difficulty">
        @foreach(['Easy', 'Medium', 'Hard'] as $level)
            <option value="{{ $level }}" @selected(old('difficulty', $course-
>difficulty) === $level)>
                {{ $level }}
            </option>
        @endforeach
    </select>
    @error('difficulty')
        <p class="error">{{ $message }}</p>
    @enderror
</div>

<div class="form-group checkbox">
    <input type="hidden" name="is_active" value="0">
    <input type="checkbox" id="is_active" name="is_active" value="1"
        @checked(old('is_active', $course->is_active))>
    <label for="is_active">Active</label>
</div>

```

The extra attributes `min` and `step` are passed straight to the `<input>` through `{{ $attributes }}` in the component. The hidden input is needed because a browser does not send an unchecked checkbox at all: with it, an unchecked box arrives as `0` and a checked box arrives as `1`.

<resources/views/course/list.blade.php>

```

@extends('layouts.app')

@section('title', 'Courses')

@section('content')
    <h1>Courses</h1>

    <a href="{{ route('courses.create') }}" class="btn">Create Course</a>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Duration</th>
                <th>Fee</th>
                <th>Difficulty</th>
                <th>Active</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @forelse($courses as $course)
                <tr>
                    <td>{{ $course->id }}</td>
                    <td>{{ $course->name }}</td>
                    <td>{{ $course->duration }} weeks</td>
                    <td>{{ number_format($course->fee, 2) }}</td>
                    <td>{{ $course->difficulty }}</td>
                    <td>{{ $course->is_active ? 'Yes' : 'No' }}</td>
                    <td>
                        <a href="{{ route('courses.show', $course) }}">View</a>
                        <a href="{{ route('courses.edit', $course) }}">Edit</a>
                        <x-delete-form :action="route('courses.destroy', $course)" />
                    </td>
                </tr>
            @empty
                <tr>

```

```

                <td colspan="7">No courses found.</td>
            </tr>
        @endforelse
    </tbody>
</table>
@endsection

```

resources/views/course/create.blade.php

```

@extends('layouts.app')

@section('title', 'Create Course')

@section('content')
    <h1>Create Course</h1>

    <form action="{{ route('courses.store') }}" method="POST">
        @csrf

        @include('course._form')

        <button type="submit" class="btn">Create Course</button>
        <a href="{{ route('courses.index') }}">Cancel</a>
    </form>
@endsection

```

resources/views/course/edit.blade.php

```

@extends('layouts.app')

@section('title', 'Edit Course')

@section('content')
    <h1>Edit Course</h1>

    <form action="{{ route('courses.update', $course) }}" method="POST">
        @csrf
        @method('PUT')
    </form>

```

```

        @include('course._form')

        <button type="submit" class="btn">Update Course</button>
        <a href="{{ route('courses.show', $course) }}">Cancel</a>
    </form>
@endsection

```

[resources/views/course/detail.blade.php](#)

```

@extends('layouts.app')

@section('title', 'Course Details')

@section('content')
    <h1>Course Details</h1>

    <p><strong>ID:</strong> {{ $course->id }}</p>
    <p><strong>Name:</strong> {{ $course->name }}</p>
    <p><strong>Description:</strong> {{ $course->description }}</p>
    <p><strong>Duration:</strong> {{ $course->duration }} weeks</p>
    <p><strong>Fee:</strong> {{ number_format($course->fee, 2) }}</p>
    <p><strong>Difficulty:</strong> {{ $course->difficulty }}</p>
    <p><strong>Active:</strong> {{ $course->is_active ? 'Yes' : 'No' }}</p>

    <a href="{{ route('courses.edit', $course) }}" class="btn">Edit Course</a>
    <a href="{{ route('courses.index') }}">Back to Courses</a>
@endsection

```

15.5 Add Courses to the navigation bar

Because the navbar exists in one file, one edit updates **every page**. Open

[resources/views/partials/navbar.blade.php](#) and add the second link:

```

<div>
    <a href="{{ route('students.index') }}"
        class="{{ request()->routeIs('students.*') ? 'active' : '' }}">
        Students
    </a>
</div>

```

```

</a>

<a href="{{ route('courses.index') }}"
    class="{{ request()->routeIs('courses.*') ? 'active' : '' }}">
    Courses
</a>

</div>

```

Checkpoint: Test list, create, view, edit and delete for Courses, including the validation errors.

Step 16: Test the Refactored Application

Restart the server and go through this list:

Test	How	Expected result
Navbar	Click Students and Courses	The current link is underlined on every page
Footer	Scroll to the bottom of any page	The footer is visible on every page
List	Open <code>/students</code>	Students are shown, or "No students found."
Create (valid)	Fill the form correctly	Redirect to the list with a green success message
Create (invalid)	Submit an empty form	Errors under each field, old input kept
Duplicate email	Create two students with the same email	Validation error, no database exception
Edit	Change a name, keep the email	Update succeeds
Delete	Click Delete and confirm	Record removed, success message shown
Missing record	Open <code>/students/9999</code>	404 page
Courses	Repeat the same tests on <code>/courses</code>	Same behavior

The DRY test: open `partials/footer.blade.php`, change the text, and refresh a Student page and a Course page. Both change, because the footer exists in one place only.

Then check the routes:

```
php artisan route:list --except-vendor
```

You should see 14 routes for `students.*` and `courses.*` (7 each).

Step 17: Push the Code to GitHub

Check the project status:

```
git status
```

Add the files and commit:

```
git add .  
git commit -m "Complete Workshop 2 MVC refactor with Blade components"
```

Push the branch:

```
git push origin Workshop2
```

Use the branch name specified in the **GitHub Guideline** if your repository uses a different naming convention.

Verify your code on GitHub.

Final Project Structure

At the end of Workshop 2, your project should contain at least:

```
workshop-project/  
|  
├─ app/  
|   ├─ Http/  
|   |   └─ Controllers/  
|   |       └─ StudentController.php  
|   |       └─ CourseController.php  
|   └─ Models/  
|       └─ Student.php  
|       └─ Course.php  
|  
├─ public/  
|   └─ css/  
|       └─ app.css  
|  
├─ resources/  
|   └─ views/  
|       └─ layouts/  
|           └─ app.blade.php  
|       └─ partials/  
|           └─ navbar.blade.php
```

```

|     |   └─ footer.blade.php
|     └─ components/
|     |   └─ alert.blade.php
|     |   └─ delete-form.blade.php
|     |   └─ form-input.blade.php
|     |   └─ form-textarea.blade.php
|     └─ student/
|     |   └─ _form.blade.php
|     |   └─ list.blade.php
|     |   └─ create.blade.php
|     |   └─ detail.blade.php
|     |   └─ edit.blade.php
|     └─ course/
|         └─ _form.blade.php
|         └─ list.blade.php
|         └─ create.blade.php
|         └─ detail.blade.php
|         └─ edit.blade.php
|
└─ routes/
    └─ web.php      (now only a few lines)
|
└─ .env
└─ composer.json
└─ artisan

```

Your final `routes/web.php` should look similar to:

```

<?php

use App\Http\Controllers\CourseController;
use App\Http\Controllers\StudentController;
use Illuminate\Support\Facades\Route;

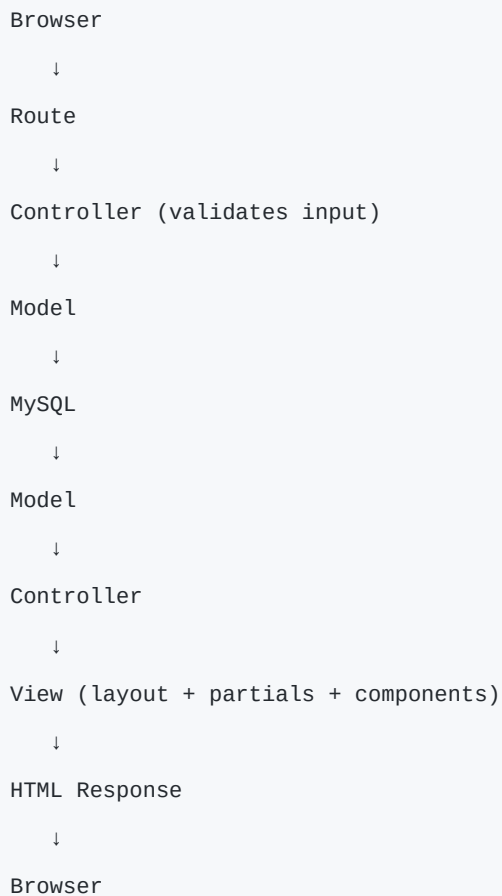
Route::get('/', function () {
    return view('welcome');
});

```

```
Route::resource('students', StudentController::class);
Route::resource('courses', CourseController::class);
```

What You Have Built

The application behaves like Week 1, but the request flow is now organized:



Workshop 2 Completion Criteria

You have completed Workshop 2 when:

- `StudentController` and `CourseController` exist with all seven resource methods.
- `routes/web.php` contains no closures for Students or Courses.
- Routes are registered with `Route::resource()`.
- All links and forms use `route()` with route names.
- Route model binding is used in `show`, `edit`, `update` and `destroy`.
- Validation rules are written once in a private `rules()` method in each controller.
- The Student model has a `date_of_birth` cast and an `age` accessor.
- All pages extend `layouts.app` and show the navbar and the footer.

- Success messages, form fields and delete buttons use Blade components.
- Create and edit share one `_form` partial for both Students and Courses.
- All Student and Course CRUD operations still work.
- Code has been committed and pushed to GitHub.

Common Errors and Fixes

Error	Likely cause	Fix
Route <code>[students.index]</code> not defined	Route not registered or not named	Check <code>Route::resource(...)</code> in <code>web.php</code> ; run <code>php artisan route:list</code>
Route <code>[courses.index]</code> not defined (on every page)	The Courses link was added to the navbar before the Course route exists	Add <code>Route::resource('courses', ...)</code> in Step 15.1 first
Target class <code>[StudentController]</code> does not exist	Missing import in <code>web.php</code>	Add <code>use App\Http\Controllers\StudentController;</code>
<code>/students/create</code> shows a 404	Explicit routes: <code>{id}</code> route defined before <code>create</code>	Define <code>/students/create</code> before <code>/students/{id}</code>
Class "Rule" not found	Missing import	Add <code>use Illuminate\Validation\Rule;</code>
View <code>[partials.navbar]</code> not found	File missing or in the wrong folder	Create <code>resources/views/partials/navbar.blade.php</code>
Unable to locate a class or view for component <code>[form-input]</code>	Component file missing or misspelled	File must be <code>resources/views/components/form-input.blade.php</code>
Page looks unstyled	<code>public/css/app.css</code> missing	Create the file and refresh with Ctrl + F5
419 Page Expired	Missing <code>@csrf</code> in a form	Add <code>@csrf</code> inside the form
Add <code>[name]</code> to fillable property	Field missing from <code>\$fillable</code>	Add the field to the model
Date of birth is empty on the edit form	The input value is not formatted	Use <code>\$student->date_of_birth?->format('Y-m-d')</code>

Error	Likely cause	Fix
Edit always says email already taken	Ignore rule not applied	Use <code>Rule::unique('students')->ignore(\$student?->id)</code> and pass <code>\$student</code> to <code>rules()</code> in <code>update</code>
Old views or components still appear	Cached data	Run <code>php artisan optimize:clear</code>